## 6. TESTING

Testing used captured PingStation replay data and local Flask execution. Python source files were compiled with `compileall`, and the dashboard JavaScript passed Node.js syntax checking. Functional tests then verified the data path from replay/live input to API output, storage, prediction and visualization.

*Table 5: Test Cases*

| Test Case | Expected Result | Observed Result |
|---|---|---|
| Python syntax | Project Python modules compile. | compileall completed successfully. |
| Dashboard syntax | Embedded JavaScript parses. | Node.js --check completed successfully. |
| Replay captured log | Saved frames are decoded. | Known aircraft records were produced. |
| Aircraft API | JSON contains normalized records. | Dashboard consumes /api/aircraft output. |
| Mode/status API | Simulation/live state is returned. | Status and runtime mode endpoints respond. |
| Coordinate filtering | Zero-position rows are excluded. | Rows with both coordinates zero are removed. |
| Trajectory storage | Latest and historical rows are stored. | SQLite contains current and trajectory tables. |